

Visual Product Similarity System

What it does, how to run it, and a full code walkthrough

Visual Product Similarity & Image-Based Recommendation System

Overview

Amazon-style visual recommendation system that finds visually similar products using only product images - upload a photo, get back the most visually similar items in the catalog.

A note on the dataset

The 58 product images shipped in `data/raw_images/` are real JPEGs, but the catalog itself is thin: checksumming them shows only **12 visually distinct images**, with two of those repeated ~20 times each under different filenames. The original metadata even assigned different product names (a shirt, a pair of sneakers, jeans) to byte-identical photos - so this was clearly built as a placeholder/demo set, not real product photography.

That means the Precision@K numbers below are real (the pipeline genuinely ran end-to-end against this data), but they're propped up by exact-duplicate images being trivially "similar to themselves" rather than by the model generalizing across genuinely different products. Swap in a real dataset (Stanford Online Products, DeepFashion, or the Amazon Product Images dataset - 1000+ images, no exact dupes) via `data/raw_images/` + `data/metadata.csv` for numbers that actually mean something.

Objective

- Extract high-quality image embeddings using a pretrained CNN
- Build a fast similarity search engine with FAISS
- Retrieve Top-K visually similar products
- Provide an optional Streamlit web app interface

Tech Stack

- Python, PyTorch
- ResNet50 (final classification layer stripped, 2048-dim pooled features)
- FAISS HNSW index (approximate nearest neighbor, inner product on L2-normalized vectors = cosine similarity)
- Streamlit

Project Structure

```
visual-product-similarity/  
|--- requirements.txt  
|--- README.md  
|--- data/  
|   |--- raw_images/      58 sample product photos (see note above)  
|   |--- processed/
```

```
| `--- metadata.csv
| --- models/          generated by the scripts below, not committed
| | --- embeddings.npy
| | --- image_paths.pkl
| | --- metadata.pkl
| | --- faiss_index.bin
| `--- src/
|   --- config.py
|   --- utils.py
|   --- extract_embeddings.py
|   --- build_index.py
|   --- search.py
|   --- evaluate.py
|   --- app.py
```

Metadata format

data/metadata.csv:

```
image_name,product_id,category,title,price,availability
img1.jpg,P001,Clothing,Blue Cotton Shirt,29.99,In Stock
img4.jpg,P002,Footwear,White Sneakers,59.99,In Stock
```

Installation

```
pip install -r requirements.txt
```

Run steps

```
python src/extract_embeddings.py # ResNet50 -> models/embeddings.npy (~1-2 min on CPU)
python src/build_index.py       # builds the FAISS HNSW index
python src/evaluate.py          # Precision@K / Recall@K over the catalog
streamlit run src/app.py        # interactive search UI
```

Results (on the shipped 58-image sample set)

```
Evaluated queries: 58
Average Precision@5: 0.3966
Average Recall@5: 0.0830
```

Given the dataset note above, read this as "the pipeline works and returns sensible, visually-grounded matches" rather than as a benchmark number - a duplicate image scores a perfect 1.0000 cosine similarity against its copies, which is exactly what you'd want, but it's not really testing generalization to unseen products.

Demo walkthrough

```
$ python src/search.py
Enter query image path: data/raw_images/img3.jpg
1. img98.jpg | Score: 1.0000
2. img93.jpg | Score: 1.0000
3. img88.jpg | Score: 1.0000
4. img83.jpg | Score: 1.0000
5. img8.jpg | Score: 1.0000
```

(img3 has 20 byte-identical copies elsewhere in the sample set, hence the perfect scores - see the dataset note.)

In the Streamlit app: upload any image from `data/raw_images/`, optionally narrow by category/price/availability in the sidebar, and the top-K panel fills in with the closest matches and their metadata.

Features

- ResNet50 embedding extraction, L2-normalized
- FAISS HNSW approximate nearest-neighbor search
- Cosine similarity ranking
- Filtering by category, price range, and availability
- Streamlit UI for image upload and result display

Evaluation

- Precision@K, Recall@K (`src/evaluate.py`, category match used as the relevance proxy)
- Visual inspection via the Streamlit results grid

Notes

- Place product images inside `data/raw_images/`; corrupt/unreadable files are skipped with a warning during extraction rather than crashing the whole run
- If metadata is unavailable for an image, the pipeline still works with just the image path
- `models/` and `data/processed/` are regenerated by the scripts, not committed to git

Visual Product Similarity & Recommendation System -- How to Run It & Code Walkthrough

This document has two parts:

1. **How to run the project**, start to finish, on a clean machine.
2. **A line-by-line explanation of every script** -- `src/config.py`, `src/utils.py`, `src/extract_embeddings.py`, `src/build_index.py`, `src/search.py`, `src/evaluate.py`, and `src/app.py`.

For the "what does the spec ask for vs. what's actually here" context (the dataset situation, the merged-branch history, the real Precision@K/Recall@K numbers), see [README.md](#). This document is purely about the code.

Part 1 -- How to Run the Project

1. Prerequisites

- Python 3.9+
- `data/raw_images/` already has the 58 sample product photos, and `data/metadata.csv` already describes all of them - nothing extra needed to run the demo as-is. To use your own catalog instead, drop images into `data/raw_images/` and write a matching `data/metadata.csv` (format in the README).

2. Install dependencies

```
pip install -r requirements.txt
```

Installs `numpy`, `pandas`, `pillow`, `tqdm`, `faiss-cpu`, `streamlit`, `torch`, `torchvision`, `scikit-learn`, `matplotlib`. The first time `extract_embeddings.py` runs, PyTorch downloads the pretrained ResNet50 weights (~98MB) to its local cache - that only happens once.

3. Run the pipeline, in this order, from the project root

```
python src/extract_embeddings.py # ResNet50 -> models/embeddings.npy
python src/build_index.py       # builds the FAISS HNSW index
python src/evaluate.py          # prints Precision@K / Recall@K
streamlit run src/app.py        # interactive search UI
```

Each script only depends on the previous one's output file, not on being run from any particular working directory in relation to `src/` - `config.py` resolves every path from its own file location, so `python src/extract_embeddings.py` works the same whether your terminal's `cwd` is the project root or somewhere else, as long as you invoke it with that relative/absolute path.

4. Using the search tool and the dashboard

`python src/search.py` run directly prompts for a query image path on the command line and prints a ranked text list - useful for a quick sanity check without opening a browser. `streamlit run src/app.py` opens the full UI at

`http://localhost:8501`: upload an image, optionally narrow by category/price/availability in the sidebar, and the results grid fills in with the closest matches.

5. If something goes wrong

Symptom	Fix
<code>ValueError: No images found in data/raw_images/</code>	Check the folder actually has <code>.jpg/.jpeg/.png/.webp</code> files in it
<code>FileNotFoundError</code> for <code>embeddings.npy</code> / <code>faiss_index.bin</code>	Run the missing earlier pipeline step
<code>ModuleNotFoundError</code>	<code>pip install -r requirements.txt</code>
Streamlit app errors on a fresh clone	Make sure you ran <code>extract_embeddings.py</code> and <code>build_index.py</code> first - <code>app.py</code> just loads their output, it doesn't compute anything itself

Part 2 -- Code Walkthrough

`src/config.py`

Every path and tunable constant the rest of the codebase imports from, in one place - nothing else in `src/` hard-codes a path or a magic number.

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

DATA_DIR = BASE_DIR / "data"
RAW_IMAGE_DIR = DATA_DIR / "raw_images"
PROCESSED_DIR = DATA_DIR / "processed"
METADATA_CSV = DATA_DIR / "metadata.csv"

MODELS_DIR = BASE_DIR / "models"
EMBEDDINGS_FILE = MODELS_DIR / "embeddings.npy"
IMAGE_PATHS_FILE = MODELS_DIR / "image_paths.pkl"
METADATA_PKL = MODELS_DIR / "metadata.pkl"
FAISS_INDEX_FILE = MODELS_DIR / "faiss_index.bin"

IMAGE_SIZE = 224
BATCH_SIZE = 32
TOP_K = 5
SUPPORTED_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp"}
```

`Path(__file__).resolve().parent.parent` is the same "resolve from this file's own location, not from the terminal's cwd" trick used in the other projects' `utils.py` - `Path(__file__)` is `.../visual-product-similarity/src/config.py`, `.resolve()` makes it absolute, the first `.parent` strips `config.py` down to `src/`, and the second strips `src/` down to the project root. Every other path constant is built from that root with the `/` operator, which `pathlib.Path` overloads to join path segments correctly on both Windows and Unix. `IMAGE_SIZE = 224` is ResNet50's expected input resolution (it was trained on 224x224 crops - feeding it a different size would still technically run but the pretrained weights wouldn't be extracting features the same model architecture originally expects). `SUPPORTED_EXTENSIONS` is a `set`, not a list, purely so the membership check in

`get_image_paths()` (`.suffix.lower()` in `SUPPORTED_EXTENSIONS`) is an $O(1)$ lookup rather than an $O(n)$ scan - meaningless at 4 entries, but the right habit either way.

`src/utils.py`

Every small, reusable helper the pipeline scripts share - image I/O, the ResNet50 feature extractor, normalization, and pickle read/write.

Lines 1-10 -- imports

```
import pickle
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from PIL import Image
from torchvision import models, transforms

from config import IMAGE_SIZE, RAW_IMAGE_DIR, SUPPORTED_EXTENSIONS
```

`pickle` serializes the image-path list and metadata records between scripts (they're plain Python lists/dicts, not numpy arrays, so pickle is simpler than another `.npy` file). `PIL.Image` handles the actual image decoding; `torchvision.models` supplies the pretrained ResNet50; `torchvision.transforms` builds the preprocessing pipeline.

Lines 13-14 -- `get_device()`

```
def get_device():
    return "cuda" if torch.cuda.is_available() else "cpu"
```

Checks once whether a CUDA GPU is available and returns the device string either way - every other function that needs a device (`build_feature_extractor`'s caller, `VisualSearchEngine.__init__`) calls this rather than duplicating the same `torch.cuda.is_available()` check, so the project runs unmodified on a GPU box or a laptop.

Lines 17-22 -- `get_image_paths()`

```
def get_image_paths(image_dir=RAW_IMAGE_DIR):
    image_paths = []
    for path in Path(image_dir).rglob("*"):
        if path.suffix.lower() in SUPPORTED_EXTENSIONS:
            image_paths.append(path)
    return sorted(image_paths)
```

`image_dir=RAW_IMAGE_DIR` as a default means every real call site (`extract_embeddings.py`) doesn't need to pass anything, while still letting a test or a one-off script point it at a different folder. `.rglob("*")` recursively walks every file under `image_dir`, including subfolders - useful if a real catalog organizes images into per-category subdirectories rather than one flat folder. `.suffix.lower()` normalizes `.JPG/.Jpg/.jpg` to the same check.

`sorted(image_paths)` matters more than it looks - without it, the order images get processed in (and therefore the order they land in `embeddings.npy/image_paths.pkl`) would depend on the filesystem's own directory-listing order, which isn't guaranteed stable across machines or even across runs on some filesystems; sorting makes the whole pipeline's output reproducible.

Lines 25-33 -- `get_preprocess_transform()`

```
def get_preprocess_transform():
    return transforms.Compose([
        transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])
```

`transforms.Compose([...])` chains three steps into one callable: resize every image to 224x224 (ResNet50's expected input size), convert the PIL image to a PyTorch tensor with values scaled to [0, 1], then normalize using the exact per-channel mean/std ImageNet was trained with. Those specific numbers (0.485, 0.456, 0.406 / 0.229, 0.224, 0.225) aren't arbitrary - they're the standard ImageNet normalization constants, and using anything else here would feed the pretrained weights input statistics they weren't trained on, quietly degrading every embedding.

Lines 36-41 -- `load_image()`

```
def load_image(image_path):
    try:
        image = Image.open(image_path).convert("RGB")
    except (OSError, ValueError) as exc:
        raise ValueError(f"Could not read image {image_path}: {exc}") from exc
    return image
```

`.convert("RGB")` matters for real-world image folders: some JPEGs are grayscale (1 channel) or CMYK, and some PNGs have a 4th alpha channel - forcing everything to 3-channel RGB up front means every image that reaches the model has a consistent shape, rather than crashing partway through a batch on the one weird file. The `try/except` catches the two realistic failure modes (`OSError` for a truncated/corrupt file, `ValueError` for something PIL can open but can't actually decode) and re-raises as a single `ValueError` with the file path baked into the message - this is what lets `extract_embeddings.py`'s dataset class catch one specific, identifiable exception type and skip just that file instead of the whole run dying on an unreadable image somewhere in a folder of a few thousand.

Lines 44-47 -- `load_and_preprocess_image()`

```
def load_and_preprocess_image(image_path):
    image = load_image(image_path)
    transform = get_preprocess_transform()
    return transform(image)
```

A convenience wrapper combining the two steps above - not actually called anywhere else in this codebase right now (`extract_embeddings.py` and `search.py` both call `load_image()` and apply their own transform separately, since they need the raw PIL image for other reasons too), but it's a reasonable one-liner to have for a script or a notebook that just wants "give me a model-ready tensor from a path" in one call.

Lines 50-57 -- `build_feature_extractor()`

```
def build_feature_extractor():
    # drop the final fc layer - we want the 2048-dim pooled features, not a
    # 1000-way ImageNet class prediction
    weights = models.ResNet50_Weights.DEFAULT
    model = models.resnet50(weights=weights)
    feature_extractor = torch.nn.Sequential(*list(model.children())[:-1])
```

```
feature_extractor.eval()
return feature_extractor
```

`models.ResNet50_Weights.DEFAULT` is torchvision's current recommended pretrained weight set (rather than pinning to a specific older weights enum, which torchvision has deprecated in favor of this pattern).

`model.children()` iterates the model's direct submodules in order - for ResNet50 that's the initial conv/pool layers, four residual "layer" blocks, an adaptive average pool, and finally the 1000-way classification `Linear` layer.

`list(model.children())[:-1]` takes every submodule *except* that last one, and `torch.nn.Sequential(*...)` glues them back together into a single callable module - the classification head is gone, so running an image through this returns the 2048-dim pooled feature vector the classifier would otherwise have consumed, which is exactly what an embedding-based similarity search needs instead of a class label. `feature_extractor.eval()` switches off dropout/batchnorm-update behavior that only matters during training - skipping this wouldn't crash anything, but embeddings could shift slightly between calls due to batchnorm running in training mode, which would silently make search results less stable.

Lines 60-64 -- `l2_normalize()`

```
def l2_normalize(vectors):
    # once these are unit-length, FAISS inner product == cosine similarity
    norms = np.linalg.norm(vectors, axis=1, keepdims=True)
    norms[norms == 0] = 1e-10 # avoid dividing an all-zero vector by zero
    return vectors / norms
```

`np.linalg.norm(vectors, axis=1, keepdims=True)` computes the Euclidean length of each row (each embedding) independently - `axis=1` means "reduce across the feature dimension, once per row," and `keepdims=True` keeps the result as a column vector (shape `(n, 1)`) rather than flattening it to `(n,)`, which is what makes the final `vectors / norms` broadcast correctly across every row. The comment explains the actual point of this function: once every vector has unit length, the dot product between any two of them equals the cosine of the angle between them - so FAISS's plain inner-product search (cheap, and what `IndexHNSWFlat(..., faiss.METRIC_INNER_PRODUCT)` computes) becomes mathematically identical to cosine similarity search, without needing FAISS to compute an actual cosine metric itself. `norms[norms == 0] = 1e-10` guards against a genuinely all-black or corrupted-to-zero image producing an all-zero embedding, which would otherwise divide by exactly zero and produce `NaNs` that would poison every downstream similarity score involving that row.

Lines 67-74 -- `save_pickle()` / `load_pickle()`

```
def save_pickle(obj, file_path):
    with open(file_path, "wb") as f:
        pickle.dump(obj, f)

def load_pickle(file_path):
    with open(file_path, "rb") as f:
        return pickle.load(f)
```

Thin wrappers so every other script writes `save_pickle(x, path)` instead of repeating the `with open(..., "wb") as f: pickle.dump(...)` boilerplate three separate times. `"wb"/"rb"` (binary mode) matter here since pickle produces a binary format, not text.

Lines 77-81 -- `load_metadata()`

```
def load_metadata(metadata_csv_path):
    if not metadata_csv_path.exists():
        return pd.DataFrame()
```

```
df = pd.read_csv(metadata_csv_path)
return df
```

Returns an empty DataFrame rather than raising if the metadata file simply doesn't exist - this is what lets the README claim "if metadata is unavailable, the project still works with image paths only": every downstream consumer of this DataFrame ([attach_metadata_to_paths](#) below) already handles an empty frame gracefully, so a missing metadata file degrades the experience (no titles/prices/categories to show or filter by) rather than crashing the whole extraction run.

Lines 84-101 -- [attach_metadata_to_paths\(\)](#)

```
def attach_metadata_to_paths(image_paths, metadata_df):
    records = []
    metadata_map = {}

    if not metadata_df.empty and "image_name" in metadata_df.columns:
        metadata_map = {
            str(row["image_name"]): row.to_dict()
            for _, row in metadata_df.iterrows()
        }

    for path in image_paths:
        image_name = path.name
        row = metadata_map.get(image_name, {})
        row["image_path"] = str(path)
        row["image_name"] = image_name
        records.append(row)

    return records
```

Builds a dict keyed by filename ([metadata_map](#)) once, up front - looking up each image's metadata by dict key is $O(1)$, which matters once this function is walking a real catalog of a few thousand images rather than 58; doing a [metadata_df\[metadata_df\["image_name"\] == name\]](#) filter inside the loop instead would be $O(n)$ per lookup, $O(n^2)$ overall. The [if not metadata_df.empty and "image_name" in metadata_df.columns](#) guard means a metadata CSV that exists but is empty, or one that's missing the expected join column, both degrade to "no metadata found for anything" rather than raising a [KeyError](#) partway through. For every image path (regardless of whether it had a metadata row), the function always attaches [image_path](#) and [image_name](#) itself - so even a completely metadata-less catalog still gets records with at least those two fields, which is what [search.py](#)'s results and [app.py](#)'s display code depend on always being present.

[src/extract_embeddings.py](#)

Turns every image in [data/raw_images/](#) into a 2048-dim embedding vector, saved alongside the matching image paths and metadata.

Lines 1-24 -- imports

```
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
from tqdm import tqdm

from config import (
    BATCH_SIZE, EMBEDDINGS_FILE, IMAGE_PATHS_FILE, METADATA_CSV, METADATA_PKL, MODELS_DIR,
)
```

```

from utils import (
    attach_metadata_to_paths, build_feature_extractor, get_device, get_image_paths,
    get_preprocess_transform, l2_normalize, load_image, load_metadata, save_pickle,
)

```

`torch.utils.data.Dataset/DataLoader` are PyTorch's standard batching machinery - writing a small `Dataset` subclass and handing it to a `DataLoader` gets you batching, and (if you ask for it) multi-process loading, for free instead of hand-rolling a batching loop over a plain list. `tqdm` wraps the dataloader iteration with a progress bar, which matters a lot once this is running over a real 1000+-image catalog instead of 58 images that finish in six seconds.

Lines 27-45 -- `ProductImageDataset`

```

class ProductImageDataset(Dataset):
    def __init__(self, image_paths, transform):
        self.image_paths = image_paths
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image_path = self.image_paths[idx]
        try:
            image = load_image(image_path)
        except ValueError as exc:
            print(f"Skipping unreadable image: {exc}")
            return None
        image = self.transform(image)
        return image, str(image_path)

```

A `Dataset` subclass only needs to implement `__len__` (how many items) and `__getitem__` (fetch one item by index) - `DataLoader` handles turning a stream of individual items into batches on top of that. `__getitem__` loads and transforms one image at a time, lazily (only when the `DataLoader` actually asks for that index), rather than loading every image into memory up front - the right approach for a catalog too large to comfortably fit in RAM all at once. The `try/except ValueError` around `load_image` is the other half of the error-handling fix mentioned in the README: `load_image()` (in `utils.py`) already turns any real read failure into a `ValueError` with the bad path in the message, and here that's caught, logged, and turned into a `return None` instead of propagating up and killing the whole `DataLoader` iteration - which is why the next function exists.

Lines 48-52 -- `_skip_unreadable()`

```

def _skip_unreadable(batch):
    batch = [item for item in batch if item is not None]
    if not batch:
        return None
    return torch.utils.data.dataloader.default_collate(batch)

```

A custom `collate_fn` - the function `DataLoader` calls to combine a list of individual `__getitem__` results into one batch. The default collate function PyTorch normally uses doesn't know what to do with a `None` in the middle of a batch (it expects every item to be a tensor/tuple of the same shape), so this wrapper filters out every `None` first (the sentinel `ProductImageDataset.__getitem__` returns for an unreadable image), and only then hands whatever's left to PyTorch's real `default_collate`. `if not batch: return None` covers the edge case where an *entire* batch turned out to be unreadable images - returning `None` for the whole batch, which the extraction loop below explicitly checks for and skips.

Lines 55-98 -- `extract_embeddings()`

```
def extract_embeddings():
    MODELS_DIR.mkdir(parents=True, exist_ok=True)

    image_paths = get_image_paths()
    if not image_paths:
        raise ValueError("No images found in data/raw_images/")

    metadata_df = load_metadata(METADATA_CSV)
    metadata_records = attach_metadata_to_paths(image_paths, metadata_df)

    transform = get_preprocess_transform()
    dataset = ProductImageDataset(image_paths, transform)
    dataloader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=False, collate_fn=_skip_unreadable)

    device = get_device()
    model = build_feature_extractor().to(device)

    all_embeddings = []
    all_paths = []

    with torch.no_grad():
        for batch in tqdm(dataloader, desc="Extracting embeddings"):
            if batch is None:
                continue
            batch_images, batch_paths = batch
            batch_images = batch_images.to(device)
            features = model(batch_images)
            features = features.view(features.size(0), -1)
            features = features.cpu().numpy()

            all_embeddings.append(features)
            all_paths.extend(batch_paths)

    embeddings = np.vstack(all_embeddings).astype("float32")
    embeddings = l2_normalize(embeddings)

    np.save(EMBEDDINGS_FILE, embeddings)
    save_pickle(all_paths, IMAGE_PATHS_FILE)
    save_pickle(metadata_records, METADATA_PKL)
    ...
```

`MODELS_DIR.mkdir(parents=True, exist_ok=True)` creates the output folder if it's missing, without erroring if it's already there. The early `if not image_paths: raise ValueError(...)` fails fast with a clear message rather than letting an empty catalog silently produce a zero-row embedding file that would confuse every later step. `metadata_records` is built *before* the model even loads - metadata attachment is cheap pandas/dict work, so there's no reason to wait until after the (comparatively slow) neural network pass to do it, and if it were going to fail (a malformed CSV, say) you'd rather find out immediately. `shuffle=False` on the `DataLoader` matters for reproducibility again - this is a feature-extraction pass, not training, so there's no reason to randomize order, and keeping it off means `all_paths` ends up in the same sorted order `get_image_paths()` produced. `with torch.no_grad():` disables PyTorch's gradient tracking for everything inside the block - this is pure inference, no backpropagation happens, so tracking gradients would only waste memory and compute for no benefit. `if batch is None: continue` is where the custom collate function's "whole batch was unreadable" case actually gets skipped. `features.view(features.size(0), -1)` flattens each sample's output from ResNet50's `(batch, 2048, 1, 1)` shape (post average-pool, pre-classifier) down to `(batch, 2048)` - a plain 2048-dim vector per image. `.cpu().numpy()` moves the tensor off the GPU (a no-op if already on CPU) and converts to a numpy array, since everything downstream (FAISS, numpy save/load) works in numpy, not PyTorch tensors. `np.vstack(all_embeddings)` stacks the list of per-batch arrays into one big `(total_images, 2048)` array; `l2_normalize` (from `utils.py`) then unit-normalizes every row so the later FAISS inner-product search behaves as

cosine similarity.

src/build_index.py

Loads the saved embeddings and builds the FAISS index used for fast similarity search.

```
import faiss
import numpy as np

from config import EMBEDDINGS_FILE, FAISS_INDEX_FILE, MODELS_DIR

def build_faiss_index():
    MODELS_DIR.mkdir(parents=True, exist_ok=True)

    embeddings = np.load(EMBEDDINGS_FILE).astype("float32")
    dimension = embeddings.shape[1]

    # HNSW graph index instead of a flat index - actual ANN search instead of
    # brute force, so this doesn't fall over once the catalog is more than a
    # few thousand images. inner product on the L2-normalized embeddings we
    # already saved = cosine similarity.
    index = faiss.IndexHNSWFlat(dimension, 32, faiss.METRIC_INNER_PRODUCT)
    index.hnsw.efConstruction = 40
    index.add(embeddings)

    faiss.write_index(index, str(FAISS_INDEX_FILE))
    ...
```

`.astype("float32")` is required because FAISS's C++ core only works with 32-bit floats, while numpy defaults to 64-bit - loading without this cast would work until `index.add()`, which would raise a dtype mismatch. `faiss.IndexHNSWFlat(dimension, 32, faiss.METRIC_INNER_PRODUCT)` builds a Hierarchical Navigable Small World graph index: the `32` is the number of graph connections per node (a standard HNSW parameter controlling the accuracy/memory tradeoff - higher means better recall but more memory and slower index-building), and `METRIC_INNER_PRODUCT` tells FAISS to rank by dot product rather than its default Euclidean distance - matching the fact that the embeddings are already L2-normalized, so inner product means cosine similarity here. `index.hnsw.efConstruction = 40` controls how thoroughly the graph is built during `index.add()` (higher = slower to build, better quality graph, better search results later) - `40` is a reasonable middle-ground default for a small-to-medium catalog; a much larger real catalog might want to tune this. This replaced an earlier version using `faiss.IndexFlatIP`, which is exact brute-force search (compares the query against literally every vector) rather than an approximate index - simpler and perfectly fine at 58 vectors, but it doesn't scale, and it doesn't technically satisfy "build an ANN index" from the project spec, which is why this switched to HNSW. `faiss.write_index(index, str(FAISS_INDEX_FILE))` serializes the whole graph structure to disk in FAISS's own binary format - `str(...)` because FAISS's Python bindings expect a plain string path, not a `pathlib.Path` object.

src/search.py

The actual query-time search logic, wrapped in a small class so the (comparatively expensive) model and index only get loaded once.

Lines 16-24 -- `VisualSearchEngine.__init__`

```

class VisualSearchEngine:
    def __init__(self):
        self.index = faiss.read_index(str(FAISS_INDEX_FILE))
        self.image_paths = load_pickle(IMAGE_PATHS_FILE)
        self.metadata = load_pickle(METADATA_PKL)

        self.device = get_device()
        self.model = build_feature_extractor().to(self.device)
        self.transform = get_preprocess_transform()

```

Everything expensive - reading the FAISS index back from disk, loading the saved paths/metadata, and rebuilding the ResNet50 feature extractor - happens once, in the constructor, rather than being repeated on every search call. This is exactly why `app.py` wraps its instantiation in `@st.cache_resource`: without that, Streamlit would re-run this whole constructor (including reloading ResNet50) on every single UI interaction, which would make the app painfully slow.

Lines 26-36 -- `extract_query_embedding()`

```

def extract_query_embedding(self, image_path):
    image = load_image(image_path)
    tensor = self.transform(image).unsqueeze(0).to(self.device)

    with torch.no_grad():
        features = self.model(tensor)
        features = features.view(features.size(0), -1)
        features = features.cpu().numpy().astype("float32")

    features = l2_normalize(features)
    return features

```

The same embedding-extraction logic as `extract_embeddings.py`, but for a single query image instead of a batched catalog pass - which is why it's duplicated here rather than shared: batching one image doesn't make sense, and pulling the batched version apart to handle a single image would be more convoluted than just writing the single-image version directly. `.unsqueeze(0)` adds a batch dimension of size 1 - the model expects input shaped `(batch, channels, height, width)`, and a single transformed image is only `(channels, height, width)` without it. The rest mirrors `extract_embeddings.py` exactly: no-grad inference, flatten to a plain vector, move to numpy, L2-normalize so it's directly comparable (via inner product) to the index's already-normalized catalog vectors.

Lines 38-78 -- `search()`

```

def search(self, query_image_path, top_k=TOP_K, category=None, min_price=None, max_price=None, availability=None):
    query_embedding = self.extract_query_embedding(query_image_path)
    scores, indices = self.index.search(query_embedding, top_k * 5)

    results = []
    for score, idx in zip(scores[0], indices[0]):
        if idx == -1:
            continue

        item = dict(self.metadata[idx]) if idx < len(self.metadata) else {}
        item["similarity_score"] = float(score)
        item["image_path"] = self.image_paths[idx]

        if category and item.get("category") != category:
            continue
        ...
        results.append(item)

    if len(results) >= top_k:

```

```

        break
    return results

```

`self.index.search(query_embedding, top_k * 5)` asks FAISS for 5x more candidates than actually needed - the comment explains why: the category/price/availability filters run *after* the FAISS search, so if the top 5 raw matches all happen to be the wrong category, without over-fetching there'd be nothing left to show. `scores, indices = ...` unpacks FAISS's return value: `indices[0]` are the row-numbers (into the original embeddings array) of the nearest matches for this one query, `scores[0]` are the matching similarity scores in the same order - the `[0]` is because FAISS's API is batch-oriented (you can search multiple queries at once), and here there's only ever one query, so only the first row of results matters. `if idx == -1: continue` - FAISS returns `-1` for a result slot when there simply aren't enough vectors in the index to fill the requested `top_k`, which happens naturally on a small catalog like this 58-image one; skipping it rather than treating `-1` as a real row index avoids an out-of-bounds lookup. `dict(self.metadata[idx])` makes a copy of the stored metadata record rather than mutating the original - important since the next two lines add `similarity_score/image_path` keys, and doing that on the shared `self.metadata[idx]` dict directly would permanently pollute it with the *last* query's score on every subsequent search. The three filter blocks (category, min/max price, availability) each `continue` past a candidate that doesn't match rather than building a separate filtered list afterward - simpler control flow, and it means the `if len(results) >= top_k: break` right after appending can stop the loop the moment enough *filtered* results are collected, without processing candidates that will never be needed. The price filters wrap the comparison in `try/except Exception: continue` because `item.get("price", 0)` could be a non-numeric value (a blank cell, a stray string) for a malformed metadata row - rather than crashing the whole search on one bad row, that row is just treated as not matching the price filter.

Lines 81-86 -- CLI entry point

```

if __name__ == "__main__":
    engine = VisualSearchEngine()
    sample_query = input("Enter query image path: ").strip()
    results = engine.search(sample_query, top_k=5)

    for i, item in enumerate(results, start=1):
        print(f"{i}. {item.get('image_name')} | Score: {item['similarity_score']:.4f}")

```

A minimal command-line way to try the search engine without opening the Streamlit app - this is what the README's demo-walkthrough example uses. `input(...).strip()` reads a path typed at the terminal, stripping any trailing whitespace/newline. `enumerate(results, start=1)` numbers the printed results starting at 1 instead of 0, matching how a person would naturally read "1st, 2nd, 3rd match" rather than "0th, 1st, 2nd."

src/evaluate.py

Computes Precision@K and Recall@K over the whole catalog, using each image's own category label as the ground-truth "what counts as a correct match" signal.

Lines 8-19 -- `precision_at_k()` / `recall_at_k()`

```

def precision_at_k(retrieved, query_category, k):
    if k == 0:
        return 0.0
    relevant = sum(1 for item in retrieved[:k] if item.get("category") == query_category)
    return relevant / k

```

```
def recall_at_k(retrieved, query_category, total_relevant, k):
    if total_relevant == 0:
        return 0.0
    relevant = sum(1 for item in retrieved[:k] if item.get("category") == query_category)
    return relevant / total_relevant
```

Both functions define "relevant" the same way: a retrieved item counts as a correct match if its category matches the query image's own category - a practical stand-in for true relevance judgments, which this project doesn't have (nobody hand-labeled "these two products are actually visually similar" pairs). Precision@K is "of the K items I retrieved, what fraction were actually relevant" - straightforward, since the denominator is just `k`. Recall@K is "of all the relevant items that exist, what fraction did I actually retrieve in my top K" - which needs to know `total_relevant` (how many other same-category items exist at all) as a separate argument, since that number depends on the whole catalog, not just what got retrieved. Both guard their respective zero-denominator case (`k == 0`, `total_relevant == 0`) by returning `0.0` rather than raising a `ZeroDivisionError`.

Lines 22-63 -- `evaluate()`

```
def evaluate(k=5, max_queries=100):
    metadata = load_pickle(METADATA_PKL)
    engine = VisualSearchEngine()

    category_counts = defaultdict(int)
    for item in metadata:
        category = item.get("category")
        if category:
            category_counts[category] += 1

    precisions = []
    recalls = []
    evaluated = 0

    for item in metadata:
        image_path = item.get("image_path")
        query_category = item.get("category")

        if not image_path or not query_category:
            continue

        results = engine.search(image_path, top_k=k + 1)
        results = [r for r in results if r.get("image_path") != image_path][:k]

        total_relevant = max(category_counts[query_category] - 1, 0)

        p_at_k = precision_at_k(results, query_category, k)
        r_at_k = recall_at_k(results, query_category, total_relevant, k)

        precisions.append(p_at_k)
        recalls.append(r_at_k)

        evaluated += 1
        if evaluated >= max_queries:
            break

    avg_precision = sum(precisions) / len(precisions) if precisions else 0
    avg_recall = sum(recalls) / len(recalls) if recalls else 0
    ...
```

`category_counts = defaultdict(int)` tallies how many catalog items exist per category up front - needed for `recall_at_k`'s denominator, and computed once here rather than recounting inside the per-query loop. The evaluation loop treats *every image in the catalog* as its own query, one at a time: `engine.search(image_path,`

`top_k=k + 1`) asks for one extra result beyond `k`, specifically because a query image is always going to find *itself* as its own nearest neighbor (a cosine similarity of exactly 1.0 against itself) - the next line, `[r for r in results if r.get("image_path") != image_path][:k]`, filters that self-match out before truncating back down to exactly `k` real results. `total_relevant = max(category_counts[query_category] - 1, 0)` subtracts 1 from the category count for the same reason - the query image itself is a member of its own category, but it can't be one of the *other* relevant items it's supposed to retrieve. `if not image_path or not query_category: continue` skips any catalog entry missing either field (nothing to search with, or nothing to score against). `max_queries=100` caps how many images actually get evaluated - on a large catalog, evaluating literally every single image as a query could take a while; capping it gives a representative sample without waiting for the whole catalog (on this project's 58-image sample set, that cap never actually triggers, since $58 < 100$). The final averages each guard against an empty `precisions/recalls` list (which would only happen if literally nothing in the catalog had both an image path and a category) with a `... if precisions else 0` conditional rather than risking a `ZeroDivisionError` on `sum([]) / len([])`.

`src/app.py`

The Streamlit dashboard - upload an image, optionally filter, see the closest visual matches.

Lines 1-9 -- imports

```
import tempfile
from pathlib import Path

import pandas as pd
import streamlit as st
from PIL import Image

from search import VisualSearchEngine
from config import METADATA_CSV
```

`tempfile` is used to write the uploaded image to a real file on disk temporarily - `VisualSearchEngine.search()` (via `extract_query_embedding`) expects a file *path* it can pass to `load_image()`, not an in-memory byte stream, so the uploaded file needs to land somewhere on disk first.

Lines 12-17 -- page setup

```
st.set_page_config(page_title="Visual Product Similarity Search", layout="wide")
st.title("Visual Product Similarity & Recommendation System")

st.write(
    "Upload a product image to find visually similar items using deep learning embeddings and FAISS."
)
```

`layout="wide"` uses the full browser width instead of Streamlit's default centered-narrow-column layout - sensible here since the results grid further down lays out multiple image columns side by side, which needs the extra horizontal space.

Lines 19-30 -- cached loaders

```
@st.cache_resource
def load_engine():
    return VisualSearchEngine()
```

```

@st.cache_data
def load_metadata():
    return pd.read_csv(METADATA_CSV) if Path(METADATA_CSV).exists() else pd.DataFrame()

engine = load_engine()
metadata_df = load_metadata()

```

`@st.cache_resource` on `load_engine` is what makes the whole app usable at all - without it, Streamlit's "rerun the whole script on every interaction" model would reload the FAISS index and rebuild ResNet50 on every single filter change or button click, since `VisualSearchEngine()`'s constructor does exactly that expensive work.

`@st.cache_data` on `load_metadata` is a smaller but real fix - this used to be a bare module-level `pd.read_csv(...)` call that re-read the CSV from disk on every rerun; caching it means the file is only actually read once per session, the same principle as the engine caching just applied to a plain DataFrame result instead of a stateful object.

Lines 32-41 -- sidebar filters

```

categories = sorted(metadata_df["category"].dropna().unique().tolist()) if "category" in metadata_df.columns
availability_options = sorted(metadata_df["availability"].dropna().unique().tolist()) if "availability" in metadata_df.columns

with st.sidebar:
    st.header("Filters")
    selected_category = st.selectbox("Category", ["All"] + categories)
    min_price = st.number_input("Min Price", min_value=0.0, value=0.0, step=100.0)
    max_price = st.number_input("Max Price", min_value=0.0, value=100000.0, step=100.0)
    selected_availability = st.selectbox("Availability", ["All"] + availability_options)
    top_k = st.slider("Top K Results", min_value=1, max_value=20, value=5)

```

Both `categories/availability_options` are built defensively - `if "category" in metadata_df.columns else []` means a metadata CSV missing that column entirely (rather than just having no rows) doesn't crash the page, it just means that filter dropdown only ever offers "All." `["All"] + categories` prepends a catch-all option to whatever real values exist, so "no filter" is always a selectable, obvious first choice rather than requiring the user to somehow select nothing.

Lines 43-64 -- running the search

```

uploaded_file = st.file_uploader("Upload a query image", type=["jpg", "jpeg", "png", "webp"])

if uploaded_file is not None:
    query_image = Image.open(uploaded_file).convert("RGB")
    st.subheader("Query Image")
    st.image(query_image, width=300)

    with tempfile.NamedTemporaryFile(delete=False, suffix=".jpg") as tmp_file:
        query_image.save(tmp_file.name)
        query_image_path = tmp_file.name

    category_filter = None if selected_category == "All" else selected_category
    availability_filter = None if selected_availability == "All" else selected_availability

    results = engine.search(
        query_image_path=query_image_path,
        top_k=top_k,
        category=category_filter,
        min_price=min_price if min_price > 0 else None,
        max_price=max_price if max_price < 100000 else None,
        availability=availability_filter,

```

```
)  
...
```

`st.file_uploader(..., type=[...])` restricts the browser's file picker to image extensions and gives back an in-memory file-like object once something's uploaded, or `None` if nothing has been uploaded yet - the `if uploaded_file is not None:` guard is what makes the whole search section only render after a real upload. `Image.open(uploaded_file).convert("RGB")` reads that in-memory upload directly with PIL (same RGB-forcing reasoning as `utils.load_image`). `tempfile.NamedTemporaryFile(delete=False, suffix=".jpg")` creates a real temp file on disk and deliberately does *not* auto-delete it when the `with` block exits (`delete=False`) - a normal `NamedTemporaryFile` deletes itself the moment it's closed, but `engine.search()` needs to open that same path again afterward (outside this `with` block), so it has to still exist by then; the OS will clean up temp files eventually regardless. `category_filter = None if selected_category == "All" else selected_category` translates the UI's "All" sentinel back into the `None` that `VisualSearchEngine.search()` actually expects for "no filter" - same pattern for availability. `min_price if min_price > 0 else None / max_price if max_price < 100000 else None` do the equivalent translation for the two number inputs: since `min_price`'s default is `0.0` and `max_price`'s default is `100000.0`, leaving either widget untouched at its default value is treated as "the user didn't actually set a price filter," passing `None` through rather than a technically-always-true bound.

Lines 66-81 -- rendering results

```
st.subheader("Similar Products")  
  
if not results:  
    st.warning("No similar products found.")  
else:  
    cols = st.columns(3)  
    for idx, result in enumerate(results):  
        col = cols[idx % 3]  
        with col:  
            st.image(result["image_path"], use_container_width=True)  
            st.write(f"**Image:** {result.get('image_name', 'N/A')}")  
    ...
```

`st.columns(3)` creates three side-by-side layout slots once; `cols[idx % 3]` cycles through them as results are rendered - result 0 goes in column 0, result 1 in column 1, result 2 in column 2, result 3 wraps back around to column 0, and so on, which is what turns a flat list of results into a 3-wide image grid rather than one long vertical list. `with col:` routes every `st.image/st.write` call for that iteration into that specific column. Every field pulled from `result` uses `.get(key, 'N/A')` rather than `result[key]` - since not every result dict is guaranteed to have every metadata field (a catalog image with no metadata row at all only has `image_path/image_name` attached, per `attach_metadata_to_paths()` in `utils.py`), so missing fields render as a plain "N/A" instead of raising a `KeyError` and crashing the whole results display over one incomplete record.